LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 10 – PENCARIAN NILAI MAX/MIN

MUHAMAD MIRZA AGUSTA

109082500044

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Praktikum Modul 10 ini berfokus pada konsep pencarian nilai ekstrem (maksimum dan minimum) dalam suatu kumpulan data menggunakan bahasa Go. Pencarian nilai ekstrem merupakan salah satu algoritma dasar yang sangat penting dalam pengolahan data, memungkinkan program untuk menemukan nilai tertinggi dan terendah beserta posisinya dalam struktur data seperti array atau slice.

A. Algoritma Pencarian Nilai Ekstrem

Algoritma pencarian nilai maksimum dan minimum bekerja dengan menginisialisasi variabel penyimpan nilai ekstrem menggunakan elemen pertama dari kumpulan data. Kemudian dilakukan perulangan dari elemen kedua hingga akhir, di mana setiap elemen dibandingkan dengan nilai ekstrem saat ini. Jika ditemukan elemen yang lebih besar dari nilai maksimum, maka nilai maksimum diperbarui. Sebaliknya, jika ditemukan elemen yang lebih kecil dari nilai minimum, maka nilai minimum diperbarui. Algoritma ini memiliki kompleksitas waktu O(n) karena data diproses satu kali secara berurutan.

B. Fungsi dengan Multiple Return Values

Go memiliki fitur yang memungkinkan fungsi mengembalikan lebih dari satu nilai (multiple return values). Fitur ini sangat berguna dalam pencarian nilai ekstrem karena fungsi dapat mengembalikan nilai yang ditemukan sekaligus indeks posisinya. Hal ini membuat kode menjadi lebih efisien dan mudah dipahami dibandingkan harus menggunakan parameter pointer atau variabel global.

C. Pointer sebagai Parameter Fungsi

Pointer adalah variabel yang menyimpan alamat memori dari variabel lain. Dalam konteks pencarian nilai ekstrem, pointer dapat digunakan untuk memungkinkan suatu fungsi memodifikasi langsung variabel di luar fungsi tersebut. Pendekatan ini berguna ketika fungsi perlu mengembalikan lebih dari dua nilai atau ketika ingin menghindari penggunaan global variabel.

D. Pencarian pada Data Terstruktur (Struct)

Pencarian nilai ekstrem tidak hanya terbatas pada tipe data sederhana seperti integer atau float, tetapi juga dapat diterapkan pada field-field dalam struct. Sebagai contoh, untuk mencari mahasiswa dengan IPK tertinggi, algoritma yang sama digunakan namun perbandingan dilakukan pada field IPK dari setiap elemen struct mahasiswa.

B. Guided

1. [Program Array mencari nilai]

```
package main

import "fmt"

func nilaiTerbesar(array []int) (int, int) {
   var idx int
   var nilaiTerbesar int = array[0]
   var idxDitemukan int = 0
   for idx = 1; idx < len(array); idx++ {
      if array[idx] > nilaiTerbesar {
         nilaiTerbesar = array[idx]
         idxDitemukan = idx
      }
   }
   return nilaiTerbesar, idxDitemukan
}

func nilaiTerkecil(array []int) (int, int) {
   var idx int
   var nilaiTerkecil int = array[0]
   var idxDitemukan int = 0
   for idx = 1; idx < len(array); idx++ {
      if array[idx] < nilaiTerkecil {
         nilaiTerkecil = array[idx]
         idxDitemukan = idx
      }
   }
   return nilaiTerkecil, idxDitemukan
}

func main() {
   var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}

   var nilaiMaks, idxNilaiMaks int =
nilaiTerbesar(arrAngka[:])
   var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])

   fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-
%d", nilaiMin, idxNilaiMin)
   fmt.Println()
   fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-
%d", nilaiMaks, idxNilaiMaks)

}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUST
MIRZA-AGUSTA\weeki 9\guided\guided 1\tempCodeRunnerFile.go"
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUST
```

Penjelasan :

Program ini mendemonstrasikan penggunaan fungsi dengan multiple return values untuk mengembalikan dua informasi sekaligus yaitu nilai ekstrem dan indeks di mana nilai tersebut ditemukan.Program mendefinisikan dua fungsi utama. Fungsi nilaiTerbesar menerima parameter slice of integer dan mengembalikan dua nilai yaitu bilangan terbesar beserta indeks kemunculannya. Fungsi ini bekerja dengan menginisialisasi nilai terbesar sebagai elemen pertama array, kemudian melakukan perulangan dari indeks 1 hingga akhir untuk membandingkan setiap elemen. Jika ditemukan elemen yang lebih besar, nilai terbesar dan indeksnya diperbarui. Fungsi nilaiTerkecil bekerja dengan prinsip yang sama namun menggunakan operator < untuk mencari nilai terkecil.

2. ["Program Pencari IPK Tertinggi dan Terendah Mahasiswa]

```
package main

import "fmt"

type mahasiswa struct {
     nama string
     nim  string
     IPK  float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
     var mhsTerbesar mahasiswa = array[0]
     var idxDitemukan int = 0
     var i int

     for i = 1; i < len(array); i++ {
          if array[i].IPK > mhsTerbesar.IPK {
               mhsTerbesar = array[i]
               idxDitemukan = i
          }
     }

     return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
     var mhsTerkecil mahasiswa = array[0]
     var idxDitemukan int = 0
     var i int

     for i = 1; i < len(array); i++ {
          if array[i].IPK < mhsTerkecil.IPK {
               mhsTerkecil = array[i]
```

```
                idxDitemukan = i
            }
        }
        return mhsTerkecil, idxDitemukan
}

func main() {
        var arrMhs arrMahasiswa
        var i, j int
        for i = 0; i < len(arrMhs); i++ {
                fmt.Println("DATA MAHASISWA INDEKS KE-", i)
                fmt.Print("Masukkan nama : ")
                fmt.Scan(&arrMhs[i].nama)
                fmt.Print("Masukkan NIM :")
                fmt.Scan(&arrMhs[i].nim)
                fmt.Print("Masukkan IPK : ")
                fmt.Scan(&arrMhs[i].IPK)
                fmt.Println("========================")
        }

        fmt.Println()
        for j = 0; j < len(arrMhs); j++ {
                fmt.Println("DATA MAHASISWA INDEKS KE-", j)
                fmt.Println("Nama : ", arrMhs[j].nama)
                fmt.Println("NIM : ", arrMhs[j].nim)
                fmt.Println("IPK : ", arrMhs[j].IPK)
                fmt.Println("========================")
        }
        fmt.Println()

        var mhsMaks, mhsMin mahasiswa
        var idxMaks, idxMin int

        mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])
        mhsMin, idxMin = IPKTerkecil(arrMhs[:])

        fmt.Println("=== DATA IPK TERKECIL & TERBESAR
MAHASISWA ===")
        fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan
NIM %s, ditemukan pada indeks ke-%d", mhsMaks.IPK,
mhsMaks.nama, mhsMaks.nim, idxMaks)
        fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan
NIM %s, ditemukan pada indeks ke-%d", mhsMin.IPK,
mhsMin.nama, mhsMin.nim, idxMin)
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-
MIRZA-AGUSTA\weeki 9\guided\guided 2\2.go"
DATA MAHASISWA INDEKS KE- 0
Masukkan nama : MIRZA
Masukkan NIM :109082500044
Masukkan IPK : 3.56
=======================
DATA MAHASISWA INDEKS KE- 1
Masukkan nama : CAESAR
Masukkan NIM :109082500099
Masukkan IPK : 3.89
=======================
DATA MAHASISWA INDEKS KE- 2
Masukkan nama : ALEXANDER
Masukkan NIM :109082500032
Masukkan IPK : 3.99
=======================

DATA MAHASISWA INDEKS KE- 0
Nama :  MIRZA
NIM :  109082500044
IPK :  3.56
=======================
DATA MAHASISWA INDEKS KE- 1
Nama :  CAESAR
NIM :  109082500099
IPK :  3.89
=======================
DATA MAHASISWA INDEKS KE- 2
Nama :  ALEXANDER
NIM :  109082500032
IPK :  3.99
=======================

=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 3.99, atas nama ALEXANDER dengan NIM 109082500032, ditemukan pada indeks ke-2IPK Terkecil : 3.56, atas nama MIRZA dengan NIM 109082500044, ditemukan p
ada indeks ke-0
```

Penjelasan :

Program ini adalah aplikasi pengelolaan data mahasiswa yang digunakan untuk mencari nilai IPK terbesar dan terkecil beserta informasi mahasiswa pemiliknya struct mahasiswa dengan field nama, nim, dan IPK, kemudian tipe arr Mahasiswa sebagai array berukuran 3 elemen. Dua fungsi utama yaitu IPK Terbesar dan IPK Terkecil bekerja dengan menerima slice of mahasiswa, menginisialisasi elemen pertama sebagai kandidat, lalu melakukan perulangan untuk membandingkan field IPK setiap elemen. Kedua fungsi mengembalikan dua nilai sekaligus yaitu struct mahasiswa yang memiliki IPK ekstrem beserta indeks kemunculannya, menunjukkan penggunaan multiple return values dalam Go. Pada fungsi main, program menggunakan dua perulangan terpisah. Perulangan pertama mengisi data tiga mahasiswa dengan meminta input nama, NIM, dan IPK dari pengguna, sementara perulangan kedua menampilkan seluruh data yang telah dimasukkan sebagai konfirmasi. Setelah data lengkap, program memanggil fungsi IPK Terbesar dan IPKB Terkecil dengan mengirimkan potongan array arrMhs[:] yang mengonversi array menjadi slice, karena fungsi dirancang untuk menerima parameter slice. Hasil pencarian disimpan dalam variabel terpisah untuk kemudian ditampilkan.

C. Unguided

1. Soal Unguided

Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar.Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

..

Jawaban :

```go
package main

import "fmt"

func main() {
   var N int
   fmt.Scan(&N)

   var berat [1000]float64

   for i := 0; i < N; i++ {
      fmt.Scan(&berat[i])
   }

   minBerat := berat[0]
   maxBerat := berat[0]

   for i := 1; i < N; i++ {
      if berat[i] < minBerat {
         minBerat = berat[i]
      }
      if berat[i] > maxBerat {
         maxBerat = berat[i]
      }
   }

   fmt.Printf("%.2f %.2f\n", minBerat, maxBerat)
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_
MIRZA-AGUSTA\weeki 9\unguided\unguided 1\tempCodeRunnerFile.go"
5 2.5 3.1 1.8 4.2 2.9
1.80 4.20
```

Penjelasan :

Program ini dimulai dengan membaca bilangan bulat N yang menunjukkan jumlah data yang akan diproses, kemudian mendeklarasikan array berat dengan kapasitas

1000 elemen bertipe float64 untuk menyimpan seluruh data berat. Setelah itu, program menggunakan perulangan for untuk membaca N buah nilai berat dari pengguna dan menyimpannya ke dalam array. Program kemudian menginisialisasi variabel minBerat dan maxBerat dengan nilai elemen pertama array sebagai titik awal perbandingan. Selanjutnya, program melakukan perulangan dari indeks 1 hingga N-1, membandingkan setiap elemen dengan nilai minimum dan maksimum saat ini. Jika ditemukan berat yang lebih kecil dari minBerat, maka nilai minimum diperbarui. Jika ditemukan berat yang lebih besar dari maxBerat, maka nilai maksimum diperbarui.

2. Soal Unguided

Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Programini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Jawaban :

```
package main

import "fmt"

func main() {
    var x, y int
    fmt.Scan(&x, &y)

    var berat [1000]float64

    for i := 0; i < x; i++ {
        fmt.Scan(&berat[i])
    }

    jumlahWadah := x / y
    if x%y != 0 {
        jumlahWadah++
    }

    var totalPerWadah [1000]float64

    idxIkan := 0
    for w := 0; w < jumlahWadah; w++ {
        for i := 0; i < y && idxIkan < x; i++ {
            totalPerWadah[w] += berat[idxIkan]
            idxIkan++
        }
    }

    for w := 0; w < jumlahWadah; w++ {
        fmt.Printf("%.2f", totalPerWadah[w])
        if w < jumlahWadah-1 {
```

```
            fmt.Print(" ")
        }
    }
    fmt.Println()

    var totalSemua float64
    for w := 0; w < jumlahWadah; w++ {
        totalSemua += totalPerWadah[w]
    }
    rataRata := totalSemua / float64(jumlahWadah)
    fmt.Printf("%.2f\n", rataRata)
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-
MIRZA-AGUSTA\weeki 9\unguided\unguided 2\2.go"
7 3
2.5 3.0 1.8 4.2 2.2 3.5 1.5
7.30 9.90 1.50
6.23
```

Penjelasan :

Program ini adalah aplikasi untuk mengelompokkan berat ikan ke dalam beberapa wadah dengan kapasitas tetap dan menghitung total berat per wadah serta rata-rata total berat semua wadah dimulai dengan membaca dua bilangan bulat yaitu x yang merupakan jumlah ikan dan y yang merupakan kapasitas maksimal setiap wadah. Kemudian program membaca x buah nilai berat ikan dan menyimpannya ke dalam array berat dengan kapasitas 1000 elemen. Program menghitung jumlah wadah yang dibutuhkan dengan membagi jumlah ikan (x) dengan kapasitas wadah (y). Jika terdapat sisa pembagian (x % y != 0), maka jumlah wadah ditambah satu karena kelebihan ikan memerlukan wadah tambahan. Proses pengelompokan dilakukan menggunakan perulangan bersarang (nested loop), di mana perulangan luar mengiterasi setiap wadah, dan perulangan dalam mengisi berat ikan ke wadah tersebut sebanyak kapasitas y atau hingga semua ikan habis. Setiap berat ikan ditambahkan ke totalPerWadah[w] menggunakan indeks idxIkan yang terus bertambah untuk melacak posisi ikan yang sudah dimasukkan. Setelah semua ikan terkelompokkan, program menampilkan total berat setiap wadah dalam satu baris dengan format dua angka desimal, dipisahkan spasi antar wadah. Selanjutnya program menjumlahkan semua total per wadah ke dalam variabel totalSemua

3. Soal Unguided

Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlumencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Jawaban :

```
package main

import "fmt"

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, N int, bMin, bMax
*float64) {

   *bMin = arrBerat[0]
   *bMax = arrBerat[0]

   for i := 1; i < N; i++ {
      if arrBerat[i] < *bMin {
         *bMin = arrBerat[i]
      }
      if arrBerat[i] > *bMax {
         *bMax = arrBerat[i]
      }
   }
}

func rerata(arrBerat arrBalita, N int) float64 {

   var total float64 = 0
   for i := 0; i < N; i++ {
      total += arrBerat[i]
   }
   return total / float64(N)
}

func main() {
   var N int
   var berat arrBalita
   var minBerat, maxBerat float64

   fmt.Print("Masukan banyak data berat balita : ")
   fmt.Scan(&N)

   for i := 0; i < N; i++ {
      fmt.Printf("Masukan berat balita ke-%d: ", i+1)
      fmt.Scan(&berat[i])
   }

   hitungMinMax(berat, N, &minBerat, &maxBerat)

   rata := rerata(berat, N)

   fmt.Printf("Berat balita minimum: %.2f kg\n", minBerat)
   fmt.Printf("Berat balita maksimum: %.2f kg\n", maxBerat)
   fmt.Printf("Rerata berat balita: %.2f kg\n", rata)
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD
MIRZA-AGUSTA\weeki 9\unguided\unguided 3\3.go"
Masukan banyak data berat balita : 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
```

Penjelasan :

Program ini adalah aplikasi untuk mengolah data berat balita dengan mencari nilai minimum, maksimum, dan rata-rata. Program ini mendemonstrasikan penggunaan array dengan kapasitas tetap, fungsi dengan parameter pointer, serta perhitungan statistik dasar. Program dimulai dengan mendefinisikan tipe arrBalita sebagai array berukuran 100 elemen bertipe float64 untuk menyimpan data berat balita. Dalam fungsi main, program meminta pengguna memasukkan jumlah data N kemudian membaca N buah nilai berat balita menggunakan perulangan for. Setelah semua data terkumpul, program memanggil fungsi hitungMinMax untuk mencari nilai minimum dan maksimum, serta fungsi rerata untuk menghitung rata-rata. Fungsi hitungMinMax menerima parameter array, jumlah data, serta dua pointer bMin dan bMax. Penggunaan pointer memungkinkan fungsi ini untuk memodifikasi langsung variabel minBerat dan maxBerat yang berada di fungsi main. Di dalam fungsi, nilai minimum dan maksimum diinisialisasi dengan elemen pertama array, kemudian dilakukan perulangan untuk membandingkan setiap elemen dan memperbarui nilai jika ditemukan yang lebih kecil atau lebih besar. Fungsi rerata bekerja dengan menjumlahkan seluruh elemen array kemudian membaginya dengan jumlah data, lalu mengembalikan hasilnya sebagai float64.

D. Kesimpulan

Berdasarkan praktikum Modul 10 tentang pencarian nilai maksimum dan minimum ini, dapat disimpulkan bahwa algoritma pencarian nilai ekstrem merupakan fondasi penting dalam pengolahan data yang dapat diterapkan pada berbagai tipe data, baik tipe sederhana seperti integer dan float, maupun tipe terstruktur seperti struct. Melalui serangkaian program guided dan unguided, praktikum ini berhasil mendemonstrasikan bagaimana algoritma pencarian nilai ekstrem diimplementasikan dengan efisien menggunakan perulangan, di mana nilai awal diinisialisasi dengan elemen pertama kemudian dibandingkan dengan elemen-elemen berikutnya untuk menemukan nilai tertinggi dan terendah beserta indeks posisinya. Pada guided pertama, program array mencari nilai terbesar dan terkecil dari sekumpulan bilangan bulat beserta indeksnya menggunakan fungsi dengan multiple return values. Guided kedua mengembangkan konsep tersebut ke data terstruktur dengan mencari mahasiswa yang memiliki IPK tertinggi dan terendah menggunakan struct dan array of struct. Kedua program ini menunjukkan bagaimana fleksibilitas algoritma pencarian nilai ekstrem dapat diaplikasikan pada berbagai jenis data.

Pada unguided pertama, program pencarian berat minimum dan maksimum anak kelinci menggunakan array float64 dengan kapasitas 1000 elemen, mendemonstrasikan implementasi sederhana pencarian nilai ekstrem. Unguided kedua mengembangkan konsep lebih lanjut dengan mengelompokkan berat ikan ke dalam wadah berkapasitas tetap, menghitung total per wadah dan rata-rata total, menggunakan perulangan bersarang dan pemrosesan data berkelompok. Unguided ketiga mengimplementasikan pencarian nilai ekstrem pada data berat balita dengan memisahkan logika perhitungan ke dalam fungsi hitungMinMax menggunakan parameter pointer dan fungsi rerata untuk menghitung rata-rata. Program ini juga menunjukkan bagaimana pointer memungkinkan fungsi untuk memodifikasi langsung variabel di luar fungsi, sehingga data minimum dan maksimum dapat dikembalikan tanpa harus menggunakan multiple return values.

Secara keseluruhan, praktikum Modul 10 ini tidak hanya mengajarkan implementasi algoritma pencarian nilai maksimum dan minimum, tetapi juga melatih kemampuan dalam memilih pendekatan yang tepat sesuai dengan kebutuhan program. Pemahaman tentang kapan menggunakan multiple return values (untuk mengembalikan dua nilai sekaligus) dan kapan menggunakan pointer (untuk memodifikasi variabel asli) menjadi keterampilan penting yang diperoleh. Selain itu, kemampuan mengelompokkan data ke dalam wadah-wadah berkapasitas tetap dan melakukan perhitungan statistik seperti total, rata-rata, serta pencarian nilai ekstrem pada data terstruktur akan sangat berguna dalam pengembangan aplikasi pengolahan data yang lebih kompleks di masa mendatang, seperti sistem manajemen inventaris, analisis data penjualan, atau aplikasi statistik lainnya.